Homework 2

Vladyslav Vasylchenko Oleh Hombosh

UCU, Artificial Intelligence

Code: https://github.com/Hombosh-dev/ucu-ai-hw-2 **Best public score:** 0.3093

1 Data and validation

Task and data. Ми розпізнаємо вісім класів (людина, велосипед, легковий автомобіль, мікроавтобус, вантажівка, триколісний велосипед, автобус, мотоцикл) у кадрах відео, знятих дроном. Оцінкою є COCO mAP@[.5:.95] з максимум 500 рамками на кадр. Набір даних з мітками (VisDrone-VID [1]) містить 63 кліпи з 55 польотів (27 047 кадрів, 1,22 млн об'єктів); тестовий набір складається з 48 кліпів (3 811 кадрів).

What we found. Об'єкти дуже дрібні. Це пов'язано зі специфікою зйомки з дрона. З цього випливає, що об'єктів на фото більше і вони щільно розміщені. Наявні моделі зазвичай вгадують до 100 об'єктів на фото. Натомість, в нас їх може бути до 180.

Також, кожен ignore-регіон був продубльований 8 разів (під кожен клас). Ми прибрали усі дублікати.

Choices that follow. Під час тренування ці регіони заповнювалися нейтральним середнім кольором ImageNet (IM_MEAN), щоб уникнути хибних покарань моделі за детекцію нерозмічених скупчень об'єктів.

Ми брали кожен третій кадр, оскільки через специфіку відео сусідні кадри майже однакові.

Кадри масштабувалися до 960 px, координати обрізалися по межах, а вироджені бокси ($<$ 1 px) відфільтровувалися для запобігання появі NaN у функціях втрат.

12 категорій VisDrone зменшено до 8 класів: схожі підкласи об'єднано (pedestrian і people у person, tricycle і awning-tricycle у tricycle), а нецільовий клас others вилучено.

На тренувальних фолдах (без витоку даних) проведено K-Means кластеризацію розмірів боксів, що дозволило адаптувати сітку якорів під дрібні об'єкти (16..256 px замість дефолтних COCO 32..512 px).

Validation. Спліт потрібно робити виключно по роликам (а не по фото з роликів), інакше матимемо data leakage. Основною метрикою для валідації є mAP, яка включає в себе AP_small та AP_large. Більшість моделей претреновані на великі бокси. Через що навіть найменші бокси не завжди покривають наші маленькі об'єкти. Тому майже вся робота - це боротьба за AP_small та намагання перетнерувати моделі на іншу специфіку кадрів.

2 Detector comparison

Усі моделі навчалися на 5 епохах.

Experiment	mAP	AP50	AP_S	AP_L	ms/frame
Faster R-CNN baseline (seed 42)	0.169	0.346	0.078	0.422	44
Faster R-CNN baseline (seed 7)	0.171	0.350	0.073	0.433	43
+ K-Means anchors 16–256	0.164	0.352	0.084	0.384	43
+ same, largest anchor 512	0.163	0.346	0.083	0.389	43
+ RoI positive fraction 0.33	0.171	0.363	0.081	0.394	43
+ GIoU box loss, weight 2	0.160	0.340	0.078	0.386	43
+ GIoU box loss, weight 5	0.162	0.343	0.078	0.389	43
Baseline (seed 7), 300 proposals	0.168	0.350	0.076	0.415	41
Baseline (seed 7), 1000 proposals	0.171	0.360	0.075	0.417	45
Baseline (seed 7), 2000 proposals	0.172	0.361	0.075	0.417	51
RetinaNet baseline (COCO anchors)	0.150	0.288	0.020	0.406	41
+ K-Means anchors 16–256	0.147	0.312	0.067	0.357	41
+ anchors + GIoU box loss	0.150	0.317	0.071	0.355	41
+ anchors, IoU thresholds 0.4/0.3	0.140	0.293	0.071	0.363	41
YOLO11s baseline	0.197	0.385	0.090	0.447	20
+ P2 (stride-4) head	0.159	0.319	0.096	0.355	24
+ P2 head, box loss weight 10	0.163	0.330	0.077	0.359	25
RT-DETRv2, 300 queries (baseline)	0.240	0.463	0.099	0.517	38
100 queries	0.230	0.441	0.090	0.508	38
500 queries	0.237	0.459	0.101	0.512	39
GIoU matching cost	0.237	0.462	0.095	0.522	38
no denoising / auxiliary loss	0.159	0.327	0.041	0.382	38
single learning rate	0.246	0.467	0.108	0.538	38

Table 1: Validation results (fold 3). Indented rows change one thing relative to the row above their block's baseline. Speeds are rough: YOLO timing includes pre-processing and NMS, the others do not. Public scores are given in Table 2.

Two-stage. Обрали Faster R-CNN як бейзлайн: RPN на першій стадії відсікає складний фон, а RoIAlign на другій уточнює координати. Тюнінгували адаптацією якорів під K-Means (16..256), збільшення частки позитивних RoI з 0.25 до 0.33 і заміна Smooth L1 на GIoU.

Faster R-CNN (pos_frac = 0.33): збільшення частки позитивних RoI повернуло mAP до 0.171 і підняло AP50 з 0.350 до 0.363.

K-Means якорі у Faster R-CNN (16..256 px): підняли AP_small (з 0.073 до 0.084), але провалили AP_large (з 0.433 до 0.384), знизивши загальний mAP до 0.164.

Збільшення ваги GIoU: не допомогло малим об'єктам ані у Faster R-CNN (падіння з 0.171 до 0.160), ані в RT-DETRv2 (з 0.240 до 0.237, з AP_small 0.099 до 0.095).

Two-stage: failure cases. Ми розібрали три кадри валідації для Faster R-CNN (поріг оцінки 0.35). Усі три показують різні причини помилок.

1. Дрібні об'єкти з висоти (uav0000279_00001_v, кадр 1): розмічено 118 об'єктів, знайдено 27, пропущено 91, хибних 30. Пропуски зосереджені там, де машини й люди дрібні й щільно стоять біля будівель та вдалині вулиці. Хибні спрацювання здебільшого це помилка локалізації: рамка стоїть поруч із машиною, але IoU нижче 0.5.

2. Щільна сцена (uav0000124_00944_v, кадр 244): розмічено 133, знайдено 46, пропущено 87, хибних лише 13. Знайдено майже всі великі машини на передньому плані, а пропущено щільну смугу дрібних машин уздовж дороги та людей і велосипеди на доріжках парку. Тут проблема в повноті, а не в хибних рамках.

3. Плутанина класів (uav0000357_00920_v, кадр 346): розмічено 82, знайдено 36, пропущено 46, хибних 30, із них 20 це плутанина класів (bicycle/motor, motor/person, van/car, car/truck). Це перехрестя з багатьма пішоходами й двоколісним транспортом; зверху силуети цих класів схожі.

One-stage anchor-based. Ми взяли RetinaNet. різниця у тому, що працює без proposals. На кожну точку мапи ознак кладуться 9 якорів, що є 100тис на кадр, і одразу після цього визначається клас та зсув рамки. Головною проблемою є те що, більшість якорів потрапляють на фон. Тому RetinaNet використовує focal loss: він зменшує вагу легких прикладів, щоб модель вчилася на складних об'єктах.

Якорі з COCO дуже великі для наших даних: 46.7% рамок не мають жодного якоря з IoU ≥ 0.5, і AP_small базової моделі лише 0.020. Ми підібрали якорі під розміри об'єктів (16..256 px), після чого метрика зросла до 0.67 (7.6% непокритих). AP_small піднявся з 0.020 до 0.067, а AP50 з 0.288 до 0.312, у той час AP_large впав з 0.406 до 0.357, тож mAP майже не змінився (0.150 $\rightarrow$ 0.147). Додавши GIoU замість Smooth L1, ми повернули 0.150 і AP_small 0.071. Зниження порогів IoU для позитивних/негативних якорів до 0.4/0.3 дало 0.140: більше якорів вважалися позитивними, і модель стала гірше відрізняти об'єкти від фону. Після підбору якорів його AP_small (0.067–0.071) наближається до Faster R-CNN (0.073–0.084), але AP_large у обох падає від того, що великих якорів стає менше

One-stage anchor-free. Ми обрали YOLO11s. Головний принцип - без якорів: кожна точка сітки передбачає клас і відстані до сторін рамки. Щоб вирішити, які точки навчати на якому об'єкті, використовується Task-Aligned Assigner (TAL): для кожного об'єкта беремо точки всередині його рамки, їм ставимо оцінку з того, наскільки добре клас і IoU передбачення збігаються з розміткою. Для дрібних щільних об'єктів це слабке місце: на карті зі stride 8 об'єкт розміром 10–16 px покривають лише 1–2 точки, тому позитивних прикладів дуже мало.

Ми спробували такі експеременти: додали голову P2 (stride 4, де карта вчетверо густіша) і підняли вагу box loss з 7.5 до 10. P2 підняла AP_small з 0.090 до 0.096, але AP_large упав з 0.447 до 0.355, а mAP з 0.197 до 0.159; з вагою 10 вийшло 0.163. Іншими словами густіша карта допомогла дрібним об'єктам, одночасно показала гірші результати на більших. Ймовірно попередньо навчені ваги підходять лише тим шарам, форма яких збігається, а нова голова з'явилась у нових шарах і зсунула індекси, але це ми так думаємо. Найкращою лишилась невикрутена базова модель (0.197).

Порівняно з anchor-based моделями вона найкраща у всіх показниках: AP_small 0.090 проти 0.073 у Faster R-CNN та 0.071 у налаштованого RetinaNet, AP_large 0.447 проти 0.433 та 0.355, і при цьому доволі швидка (20 мс/кадр проти 43 і 41).

Query-based. Обрали RT-DETRv2, бо він не використовує якорі та NMS, що важливо для щільних сцен до 180 об'єктів. Тюнінгували зміною кількості запитів (100 / 300 / 500), вагами GIoU, вимкненням denoising+aux loss та LR бекбону.

RT-DETRv2: підняття lr_backbone у 10 разів (з 1e-5 до 1e-4) дало найкращий результат 0.246 mAP та 0.108 AP_small. Згорточним фільтрам ResNet-50vd, претренованим на наземному COCO, не вистачало 1e-5 для адаптації під фото з дронів за 5 епох.

Кількість запитів (q=300 vs q=500): результати практично однакові в межах шуму (0.240 vs 0.237), але q=500 трохи повільніший (39.0 проти 37.9 мс/кадр).

У щільних сценах RT-DETRv2 (q=300) виграє у Faster R-CNN (59.6% проти 45.4%) завдяки відсутності NMS, а за розміром найбільший приріст на 16-32 px (46.7% проти 32.2%) та 32-96 px (70.1% проти 57.7%).

RT-DETRv2 q=100: mAP впав до 0.230, а Recall у щільних кадрах - з 59.6% до 54.5%, бо за EDA в кадрі буває до 180 об'єктів і 100 слотів фізично не вистачає.

Вимкнення Denoising + Aux loss (exp3): обвал mAP до 0.159 і AP_small до 0.041 - без цих механізмів модель не встигає зійтися за 5 епох.

За 5 епох і Faster R-CNN (0.157 $\rightarrow$ 0.174), і базовий RT-DETRv2 (0.253 $\rightarrow$ 0.239) виходять на плато, тож перевага трансформера не є наслідком недовченості CNN. Натомість exp3 без денойзингу так і не вийшов на плато (0.072 $\rightarrow$ 0.159).

RT-DETRv2 швидший на інференсі ($\sim$38 vs $\sim$43 мс/кадр у Faster R-CNN), але тренується приблизно в 1.5 рази повільніше (16.7 vs 11.4 хв/епоху на Tesla T4).

Для об'єктів <16 px обидві архітектури знаходять лише 10-14% (обмеження роздільності stride=8).

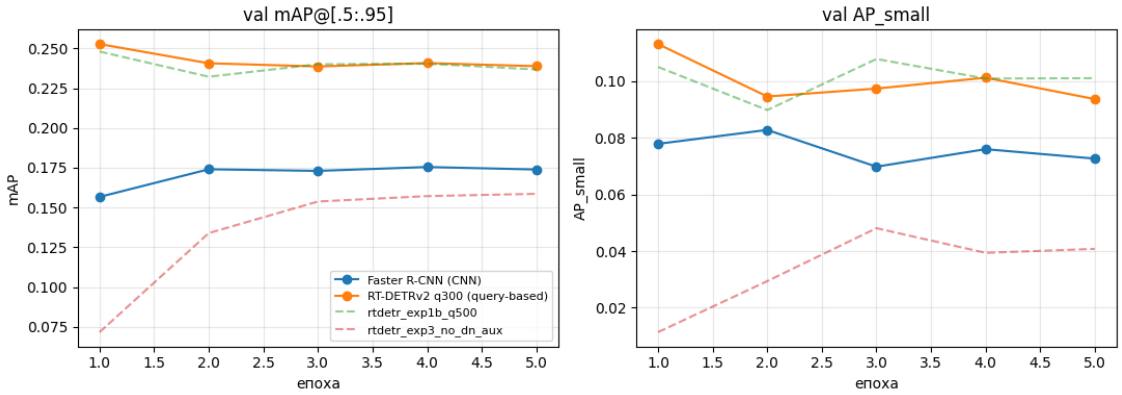


Figure 1: Validation mAP (left) and AP_S (right) per epoch, same 5-epoch budget. Faster R-CNN and RT-DETRv2 (300 and 500 queries) have flattened by epoch 5; the run without denoising and auxiliary loss is still rising.

3 Final pipeline

Data and inference strategy. Розділ 1 показав головну проблему: при стисненні кадру до 640 px 53.6% об'єктів стають меншими за 16 px , а розділи 2–5 показали, що маленькі об'єкти тягнуть вниз AP. Тому в фінальному пайплайні ми міняли не архітектуру, а дані та інференс. Модель YOLO26m ми навчали на «плитках» 640×640 у нативному масштабі: дрібна машина лишається такого самого розміру в пікселях. Плитка потрапляє в датасет, якщо видно не менше 45% об'єкта, а з порожніх плиток лишаємо лише 5%. Додатково йдуть цілі кадри, стиснуті до 640 px, щоб модель бачила цілий кадр. Рідкісним класам даємо більшу вагу в функції втрат. Усього 50 984 зображення і 983 103 рамок.

Кожен кадр проганяємо через модель у кількох варіанціях: цілий у 960, 1280 і 1536 px, віддзеркалений, збільшений, і плитками. Рамки з усіх виглядів зливаємо через Weighted Boxes Fusion (WBF): рамки навколо одного об'єкта усереднюються у одну замість того, щоб відкидати всі, крім однієї, як робить NMS. Набір виглядів вибирали жадібно: додаємо вигляд, лише якщо він піднімає mAP не менш ніж на 0.0005. Окремо підібрали IoU-поріг злиття 0.7.

Step	Val mAP	Public
YOLO11s baseline, 960 px (Sec. 4, fold of 13 clips)	0.197	0.175
YOLO26s, 1280 px, views + WBF (first pipeline)	—	0.195
YOLO26m on tiles, single full view 640 px	0.164	—
same model, full view 960 / 1280 / 1536 px	0.190 / 0.198 / 0.204	—
native tiles alone / up-scaled tiles alone	0.158 / 0.181	—
WBF: full_up + tiles_up + tiles_flip + tiles	0.240	0.250
WBF: full_1536 + tiles_up + tiles_flip + full_1280_flip, IoU 0.7	0.242	0.256
+ neighbour-frame rescoring ($\alpha = 0.2$)	0.241	not submitted
Final submission	—	**0.3093**

Table 2: Pipeline steps. Rows from YOLO26m onward use the 12-clip validation fold (see Section 1), so they are not exactly comparable with the rows above. No validation score is recorded for the final submission.

What helped and what did not. Більшість виграшу дав інференс, а не навчання. Одна й та сама модель дає 0.164 на цілому кадрі 640 px, 0.190 при збільшенні до 960, 0.198 при 1280 і 0.204

при 1536 px; плитки самі по собі слабші. Злиття виглядів через WBF піднімає mAP до 0.240, а підбір виглядів та IoU-порогу до 0.242 (AP_S 0.100, AP_L 0.411); на публічному лідерборді це 0.250 і 0.256. Найкраще визначаються bus (AP 0.524) і car (0.483), гірше за все bicycle (0.038) і truck (0.114).

Не допомогло перескорювання оцінок за сусідніми кадрами: з $\alpha = 0.2$ mAP 0.241, нижче за 0.242 без нього, тому ми його не використовуємо. Обмеження: вигляди й пороги підбирались на тому самому валідаційному фолді, що й оцінювали, тож 0.242 трохи оптимістичне; фолд у цьому розділі має 12 кліпів, а в розділах 2–5 їх 13, тому числа не зовсім порівнянні; модель, навчена на всіх даних (train і val разом), не навчена через ліміт GPU.

4 Final model and how to run it

Обране рішення: YOLO26m, навчена на вирізках 640×640 у нативному масштабі та цілих кадрах, 20 епох, batch 16; на інференсі злиття кількох виглядів кадру через WBF. Ноутбук відтворює сабміт submission_v2_val.csv.

Середовище: Kaggle/Colab, GPU Tesla T4, Python 3.13, PyTorch 2.11, Ultralytics 8.4.207, а також ensemble-boxes, pycocotools, scikit-learn, scipy, kagglehub. Seed 42, 5 фолдів, валідаційний фолд 3. Якщо сесія обривається, повторний запуск тренування продовжує з last.pt.

1. Відкрити notebooks/AI_HW2.ipynb у Kaggle або Colab, увімкнути GPU і додати дані змагання multiple-object-video-detection-hw-2-ucu-ai-2026.
2. Запустити розділ 0 (Setup), потім розділ 6 по порядку: 6.1 (залежності, фолди, метрика), 6.2 (вирізки), 6.3 (тренування), 6.4 (інференс і підбір виглядів).
3. Запустити 6.5: клітинка зберігає submission_v2_val.csv

5 Conclusions

Невеликі об'єкти та сцени з високою щільністю є головною перепоною нашої роботи та роботи моделей. RT-DETRv2 на основі запитів показав найкращі результати при однакових умовах (0,246), YOLO11 виявився найшвидшим і найефективнішим серед CNN-детекторів, а налаштування якорів лише погіршило точність розпізнавання великих об'єктів на користь точного розпізнавання невеликих. Найбільше підвищення продуктивності в кінцевому конвеєрі було досягнуто завдяки збільшенню масштабу та об'єднанню повноекранного та фрагментованого переглядів. Кожен експеримент ми запускали лише з одним seed, тому різницю менш ніж ∼0.003 mAP вважаємо шумом. Остаточну модель ми не навчали на всіх даних (train і val разом), бо не вистачило ліміту GPU

Sources of code and ideas. Detectors and training: torchvision , Ultralytics, Hugging Face Transformers ; evaluation: pycocotools; fusion: `ensemble-boxes`; folds: scikit-learn.